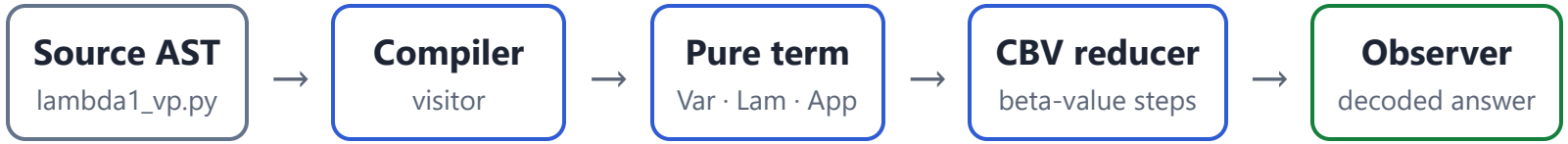
CS413 · FALL 2026 · MIDTERM 1 (ASSIGNMENT 6)

Compiling Extended LAMBDA into Pure Lambda-Calculus

Numbers, booleans, pairs, `let`, `if`, and recursion, translated into nothing but variables, one-parameter lambdas, and applications

Arnav Swamy



OVERVIEW

The goal

Translate programs, don't evaluate them and encode the answers.

- Input: an extended-LAMBDA **source AST** built in Python (no parser needed)
- Output: a term in Church's **pure, untyped λ-calculus**
- Run the output with a **bounded, weak, left-to-right call-by-value** reducer
- **Decode** the result with pure observation contexts
- Check it against the **same evaluator** run on the source, and against an explicit expected answer

Ground rules from the spec

- Compilation never calls the evaluator
- Generated code contains no source primitives: every number and operator is a λ-term
- The reducer never recognises Church encodings or uses Python arithmetic
- One evaluator handles both languages

`fact 4` is computed purely by beta reduction, yet decodes to **24**.

LANGUAGES

Source language

The supplied AST from Assignment 5, used unchanged

Constructor	Meaning
`D0Eint(n)`	natural number, n ≥ 0
`D0Ebtf(b)`	true / false
`D0Evar`, `D0Elam`, `D0Eapp`	variables, functions, application
`D0Eif0(c, t, f)`	boolean conditional
`D0Elet(x, e1, e2)`	non-recursive binding
`D0Epair`, `D0Epfst`, `D0Epsnd`	pairs and projections
`D0Efix(f, x, e)`	recursive function
`D0Eop1("not", e)`	negation
`D0Eop2(op, e1, e2)`	binary operators →

Operators

`+` `-` `*` `<` `<=` `>` `>=` `==` `!=` `and` `or`

- `-` is truncated: `2 - 5 = 0`
- `and` / `or` short-circuit left to right

Semantics

- Left-to-right **call-by-value** everywhere
- `let` binds only in its body, not its initializer
- `fix f(f)` : the parameter shadows the function name
- Negative literals, `/` , and other operators are **compile errors**, even in dead code

LANGUAGES

Target language: pure λ-calculus

A separate AST with exactly three forms (`target.py`)

```
Term ::= Var(name)
       | Lam(name, Term)
       | App(Term, Term)
```

Printed as `x` , `(lam x. body)` , `(fn arg)` .

- Immutable (frozen dataclasses)
- Free variables cached per node
- Unchanged subterms are **shared** by substitution, not copied
- `alpha_equiv` and `is_pure` for testing

Capture-avoiding substitution

```
(lam y. x)[x := y]
  = lam y_1. y        ✓ y stays free
  ≠ lam y. y          ✗ captured
```

Renaming policy (from my Assignment 5): try `y_1` , `y_2` , … and take the first name not used anywhere in the term, the replacement, or the target.

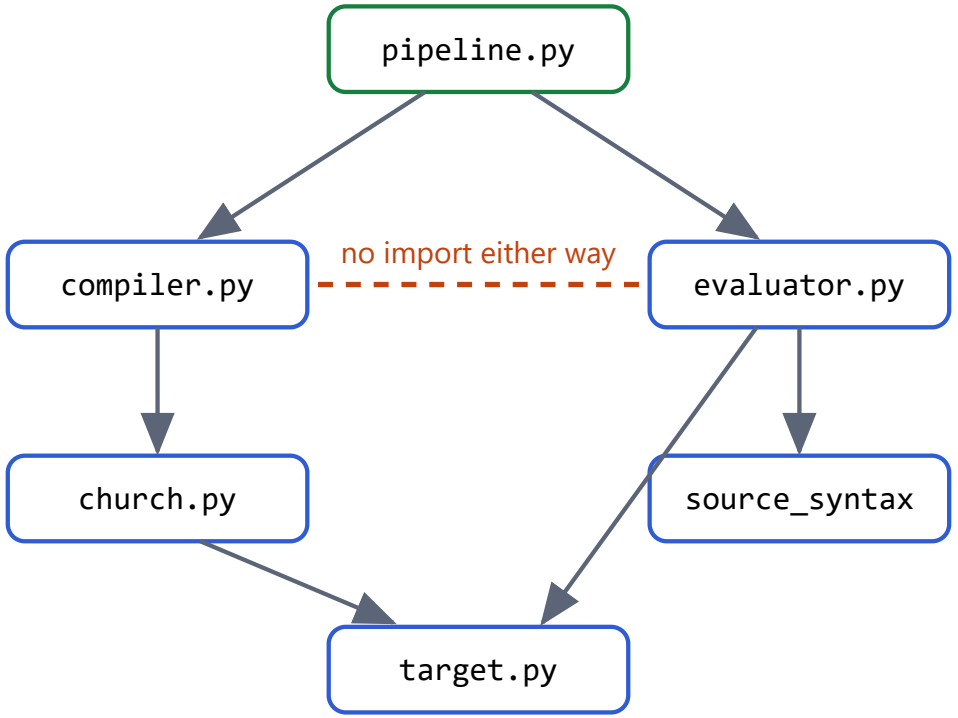
During closed execution every substituted value is closed, so renaming **never** happens there, and it costs nothing.

ARCHITECTURE

Modules and dependencies

Compiler, encodings, evaluator, and demo are kept separate

Module	Role
`target.py`	Var/Lam/App, printer, FV, substitution
`church.py`	closed Church-encoding combinators
`source_syntax.py`	source AST import, printer, substitution
`compiler.py`	`CompileVisitor`, `compile_expr`
`evaluator.py`	the one shared CBV machine
`observe.py`	nat / bool / pair decoders
`pipeline.py`	compile → execute → observe
`examples.py`, `demo.py`	demonstration programs and CLI
`TEST/`	113 unittest tests



The evaluator imports neither the compiler nor the Church library; the compiler never imports the evaluator. A test parses the imports to check this.

EVALUATION

One evaluator for both languages

The machine is a `D0ExpVisitor`; target terms speak the same `accept` protocol

```
class _Machine(D0ExpVisitor):
    # shared by BOTH languages
    def visit_var(self, node): ...  # unbound -> error
    def visit_lam(self, node): ...  # a value
    def visit_app(self, node): ...  # fn, arg, beta

    # source-only constructors, source rules
    def visit_int / visit_op2 / visit_if0 /
        visit_let / visit_pair / visit_fix ...
```

- `Var` / `Lam` / `App` mirror the fields of `D0Evar` / `D0Elam` / `D0Eapp`
- A target term can never reach a source-only method
- Iterative: a continuation stack, no Python recursion

Weak, left-to-right call-by-value

```
(lam x. body) v  -->  body[x := v]
```

- Evaluate the function, then the argument, then substitute
- Never reduce under a lambda; no eta

Step budget

- Checked **before** every beta-value step
- `I` with budget 0 → value, 0 steps
- `I I` with budget 0 → exhausted, 0 steps
- `I I` with budget 1 → value, 1 step
- Exhaustion ≠ proof of divergence

CHURCH ENCODINGS

Data as functions

Every combinator in `church.py` is a **closed** pure term

Booleans

```
TRUE  = lam t. lam f. t
FALSE = lam t. lam f. f
NOT   = lam b. b FALSE TRUE
```

A boolean *selects* one of two arguments.

Naturals

```
NUM(0) = lam s. lam z. z
NUM(2) = lam s. lam z. s (s z)
SUCC = lam n. lam s. lam z.
         s (n s z)
```

n means "apply `s` n times".

Pairs

```
PAIR = lam a. lam b. lam p.
         p a b
FST  = lam p. p TRUE
SND  = lam p. p FALSE
```

A pair hands both parts to a selector.

Names like `TRUE` and `PAIR` are only Python names. In compiled output they are expanded into full λ-terms. **NUM(0) and FALSE are the same term**, so decoding needs an expected kind.

Arithmetic, comparisons, and the predecessor

```
ADD = lam m. lam n. lam s. lam z. m s (n s z)
MUL = lam m. lam n. lam s. m (n s)
SUB = lam m. lam n. n PRED m      (truncated)
IS_ZERO = lam n. n (lam x. FALSE) TRUE

LEQ m n = IS_ZERO (SUB m n)
GEQ m n = LEQ n m
LT  m n = NOT (LEQ n m)
GT  m n = NOT (LEQ m n)
EQ  m n = AND (LEQ m n) (LEQ n m)
NEQ m n = NOT (EQ m n)
```

Every comparison reduces to "is `m − n` zero?"

Predecessor: Kleene's pair trick

A numeral can apply a function n times, but it can't un-apply one. So count **up**, carrying the previous value:

```
STEP' (a, b) = (b, b + 1)
PRED n = FST (n STEP' (0, 0))
```

steps run	pair	FST
0	`(0, 0)`	**0** ← pred 0 = 0
1	`(0, 1)`	0
2	`(1, 2)`	1
3	`(2, 3)`	2

Under CBV each step runs eagerly, so the cost is linear in n.

COMPILATION

A compositional visitor

`CompileVisitor`: compile the children, then combine them by the constructor

```
C[n]                 = NUM(n)
C[true] / C[false]   = TRUE / FALSE
C[x]                 = x
C[lam x. e]          = lam x. C[e]
C[e1 e2]             = C[e1] C[e2]
C[let x = e1 in e2]  = (lam x. C[e2]) C[e1]
C[pair e1 e2]        = PAIR C[e1] C[e2]
C[fst e] / C[snd e]  = FST C[e] / SND C[e]
C[not e]             = NOT C[e]
C[e1 op e2]          = OP C[e1] C[e2]
                       op ∈ + - * < <= > >= == !=
C[if c then t else f] = (C[c] (lam u. C[t])
                              (lam v. C[f])) I
C[e1 and e2] = (C[e1] (lam u. C[e2]) (lam v. FALSE)) I
C[e1 or e2]  = (C[e1] (lam u. TRUE) (lam v. C[e2])) I
C[fix f(x). e]       = Z (lam f. lam x. C[e])
```

- **let** is just an application, so its initializer is evaluated first and is outside the binder's scope
- Operands are arguments, so they are evaluated left to right
- **Unsupported forms** raise `CompileError` wherever they appear, including unselected branches and functions that are never called:

```
unsupported binary operator '/' in (4 / 2)
unsupported negative integer literal -3
```

> A diverging program like `LOOP` compiles normally, because compilation never runs code.

Delayed branches

Naive: apply the boolean to both branches

```
C[if c then t else f] = C[c] C[t] C[f]   ✗
```

Under CBV, arguments are evaluated **before** the call, so **both** branches run. `if true then 1 else LOOP` would never finish.

Fix: thunks, then force only the chosen one

```
(C[c] (lam u. C[t]) (lam v. C[f])) I
```

Lambdas are already values, so passing them evaluates nothing. The boolean picks one thunk, and applying it to `I = lam w. w` runs only that branch.

Program	Result
`if true then 1 else LOOP`	1
`false and LOOP`	false
`true or LOOP`	true
`if true then LOOP else 1`	exhausted
`(lam x. 0) LOOP`	exhausted
`let x = LOOP in 2`	exhausted
`fst (pair 2 LOOP)`	exhausted

Only control constructs are delayed. Function arguments, `let` initializers, and pair components remain **strict**, exactly as in the source.

CALL-BY-VALUE COMPILATION

Recursion with the Z combinator

Y loops forever under CBV

```
Y = lam g. (lam x. g (x x)) (lam x. g (x x))
```

`x x` is an argument, so it is evaluated *before* `g` gets control, again and again. A test shows that `Y G` exhausts its budget.

Z delays the self-application

```
Z = lam g. (lam x. g (lam v. x x v))
           (lam x. g (lam v. x x v))
```

`lam v. x x v` is a value: it unfolds only when it is called.

```
C[fix f(x). e] = Z (lam f. lam x. C[e])

Z G  -->*  G F           F = lam v. X X v
     -->   lam x. C[e][f := F]      ← a value

F a  -->*  (lam x. C[e][f := F]) a
     -->   C[e][f := F][x := a]
```

This matches the source rule `e[f := fix f(x). e][x := a]`.

> **Coincident binders**: `fix f(f). f` becomes `Z (lam f. lam f. f)`. The inner binder shadows the outer one, so `(fix f(f). f) 3` = **3**.

SCOPE

Names, freshness, and capture

1 · Library combinators are closed

The compiler only *applies* them to compiled code and never wraps compiled code in their binders. So `s`, `z`, `t`, `f`, `x`, … stay available to source programs.

2 · Source binders are copied as written

Shadowing, let-initializer scope, and `fix f(f)` come out right automatically.

3 · Thunk parameters are fresh

Use `u` / `v` unless that name occurs anywhere in the wrapped term, then `u_1`, `u_2`, … (deterministic).

```
if c then u else (v v_1)
  ⇒ (((c (lam u_1. u))
         (lam v_2. (v v_1)))
       (lam w. w))
```

Free variables are preserved:

$$FV(C[e]) = FV(e)$$

- Checked on 400 random **open** source ASTs
- Source programs that use every library binder name (`let s = 2 in s * 3 + s`, …) still give the right answers

DECODING

Observing results without cheating

Pure observation contexts, run by the same CBV engine with their **own** budget

Natural

```
n STEP BASE
STEP = lam tail. lam c.
         lam z. c tail
BASE = lam c. lam z. z
```

CBV builds a finite spine. Python counts the nodes, up to alpha-renaming.

Boolean

```
b MARK_T MARK_F
MARK_T = lam p.lam q.lam r. p
MARK_F = lam p.lam q.lam r. q
```

Python checks which marker came back.

Pair

```
FST p  →  observe(kind1)
SND p  →  observe(kind2)
```

Evaluate each projection under the budget, then decode recursively. `pair (pair 2 true) 3` → **((2, true), 3)**

Status is one of **ok**, **exhausted**, or **malformed**. Results like `ADD 2 3` still contain redexes inside lambdas, so matching their structure directly would fail. The contexts make them reduce instead.

WALKTHROUGH

Example: `2 + 3 * 4`

```
(((lam m. (lam n. (lam s. (lam z. ((m s) ((n s) z)))))) (lam s. (lam z. (s (s z))))) (lam s. (lam z. (s (s z)))))
   (((lam m. (lam n. (lam s. (m (n s))))) (lam s. (lam z. (s (s (s z)))))) (lam s. (lam z. (s (s (s (s z))))))))
```

- The outer application is **ADD** applied to `NUM(2)`, then to the compiled `3 * 4`
- The inner part is **MUL** applied to `NUM(3)` and `NUM(4)`
- 52 nodes, pure: only Var / Lam / App

Source evaluator	**14**
Execution	value after **4** steps
Observation (nat)	ok after **25** steps
Decoded / expected	**14 / 14** PASS

Execution is only 4 steps because `ADD` and `MUL` return lambdas right away. The real counting happens during observation.

WALKTHROUGH

A recursive program: factorial 4

```
fix fact(n). if n == 0 then 1 else n * fact (n - 1)        applied to 4
```

Compiles to `Z (lam fact. lam n. …)` applied to `NUM(4)`:

```
(((lam g. ((lam x. (g (lam v. ((x x) v)))) (lam x. (g (lam v.
((x x) v)))))) (lam fact. (lam n. ((((((lam m. (lam n. (((lam p.
(lam q. ((p q) (lam t. (lam f. f))))) (((lam m. (lam n. ((lam n.
((n (lam x. (lam t. (lam f. f)))) (lam t. (lam f. t)))) ...

          ... 1,100 more characters ...

... (lam s. (lam z. z))) (lam s. (lam z. z)))))))) m))) n)
(lam s. (lam z. (s z)))))))) (lam w. w))))) (lam s. (lam z.
(s (s (s (s z)))))))
```

Start: `Z` applied to `lam fact. lam n. …`, with `EQ` expanded. End: the dummy `(lam w. w)` that forces the chosen branch, then the argument `NUM(4)`. The full term is 1,542 characters and 344 nodes, saved in `generated/DEMO-10.txt`.

Source evaluator	**24**
Execution	**471** beta-value steps
Observation (nat)	**218** steps
Decoded / expected	**24 / 24** PASS

- `n == 0` is `EQ`: two `LEQ`s built on truncated subtraction
- The `if` keeps the recursive call inside a thunk, so the base case stops recursion
- Fibonacci 5 → **5** in 2,021 + 45 steps

RESULTS

All required demonstrations

`python MySolution/demo.py` : execution budget 10,000 and observation budget 10,000 beta-value steps

ID	Program	Exec steps	Obs steps	Decoded	Expected	
01	`2 + 3 * 4`	4	25	14	14	✓
02	`2 - 5`	87	2	0	0	✓
03	`if 2 <= 3 then 7 else 9`	77	9	7	7	✓
04	`let x = 3 in (lam y. x + y) 4`	4	13	7	7	✓
05	`fst (pair 2 true)`	6	4	2	2	✓
06	`snd (pair 2 true)`	6	2	true	true	✓
07	`(lam f. f 3) (lam x. x + 1)`	4	10	4	4	✓
08	`if true then 1 else LOOP`	3	3	1	1	✓
09	`(lam x. 0) LOOP`	10,000	n/a	exhausted	exhaustion	✓
10	factorial 4	471	218	24	24	✓
11	Fibonacci 5	2,021	45	5	5	✓
12	`LOOP`	10,000	n/a	exhausted	exhaustion	✓
13	`true and false`	3	2	false	false	✓
14	`false or true`	3	2	true	true	✓

Plus 6 extra examples (nested pairs, `fix f(f)` , short-circuiting, `let x = LOOP in 2` , `not`): **20 / 20 match**. The whole demo runs in under a second.

TESTING

Automated tests: 113 `unittest` tests, all passing

Obligation	Tests
Substitution with shadowing and capture	`test_target`
CBV order, zero-step values, budgets, determinism	`test_evaluator`
Every encoding, comparison outcome, truth table	`test_church`
Every constructor and operator, scope, fix binders	`test_compiler`
Divergence: skipped vs. strict positions	`test_demos`
Decoders: alpha-renamed, malformed, exhausted	`test_observe`
Purity, closed combinators, no evaluator calls	`test_compiler`, `test_church`

Beyond the hand-written cases

- **Differential:** 120 random well-typed programs. The compiled answer must equal the shared evaluator's source answer
- **Property:** 400 random open ASTs, checking FV preserved, pure output, input unchanged, deterministic
- **No evaluation while compiling:** evaluator entry points replaced by doubles that fail if called
- **Mutation check:** compiling `-` as `+`, or reusing a non-fresh thunk name, makes the tests fail

CORRECTNESS

Why the translation preserves meaning

Claim: for closed, well-typed programs returning naturals, booleans, or finite pairs, observing `C[e]` gives the source answer, given enough budget

- **Application:** `C[e1] C[e2]` evaluates function, then argument, then substitutes, the same order as the source. Substitution avoids capture.
- **Let:** the same argument, since it compiles to an application. The initializer stays outside the binder.
- **Conditional:** thunks are values. The boolean selects one and `I` forces it, so the other is never run.
- **Recursion:** `Z G` is a lambda value, and each call unfolds exactly one step, matching the source `fix` rule.

Testing is not proof.

- Tests cover finitely many programs
- Exhausting the budget is evidence of divergence, not proof
- Observers check one result at one kind

A full proof would use induction over evaluation and a substitution lemma for `C`. DESIGN.md sketches it.

LIMITATIONS

Limitations and future work

- **Unary numerals and substitution are slow:** `EQ` / `LEQ` cost O(m·n) steps, and larger factorials need much larger budgets
- **Closed terms only** for execution and decoding. Open terms are rejected with `EvalError`
- **No type checking:** `1 + true` compiles, and its result means nothing
- **Decoding checks shape only:** for example, `lam x. x` decodes as the number 1
- **Printed terms are long:** each combinator is expanded in full, so terms are saved to files

Next steps

- Binary or Scott numerals for cheaper arithmetic
- An environment-based machine for speed
- A source parser so examples can be written as text
- Signed integers and lists as separately tested extensions

SUMMARY

Every number, boolean, pair, branch, and recursive call became a λ-term

20/20
demonstrations match

113
tests passing

0
source primitives in the output

```
cd assigns/06
python -m unittest discover -s MySolution/TEST -v
python MySolution/demo.py
```